CA1 - DESIGN & CREATE

Public Speaking Practice Evaluator

CAP455 - Object-Oriented Programming using C++

STUDENT DETAILS	
Name	Saurabh kumar
Roll / Registration No.	12611227
Section	D1P2635
Report date	06 November 2026

GitHub repository: https://github.com/Saurabh8820/CPP-CA

CA1 project report

A C++ console application that stores speaking-practice sessions, calculates feedback from manually supplied counts, and supports validated Create and Read operations. Each record contains a unique session ID, speaker name, speech topic, duration, word count and filler-word count.

CONTENTS	PAGE
Project proposal and data dictionary	2
UML and class features	3
Program flowchart and Create algorithm	4
Arrays, pointers, strings and evaluation logic	5
Normal, invalid and boundary tests	6
Complete source code	7-9
Execution results and conclusion	10

Implemented features

Entity class with static, inline and friend features; an array of objects; object pointers; strings; validated record creation; duplicate-ID prevention; and retrieval of all records or one record by ID.

Scope: one entity class, a fixed object array, unique session IDs, validation, and data held in memory for the current run.

CA1 / PROJECT REPORT

1. Proposal and data dictionary

Problem and proposed solution

Students often practise speeches without keeping consistent records of duration, word count and filler-word usage. Scattered notes make it difficult to locate a particular practice attempt. The proposed application keeps an organised practice log and provides repeatable, basic feedback for each stored session.

The user manually enters a speaker name, topic, duration, total words and filler words after a practice attempt. The program assigns the supplied unique session ID, validates the details, and calculates speaking rate and filler percentage when a record is displayed.

PROPOSAL ITEM	DESIGN
Primary entity / main class	One speaking-practice attempt / PracticeSession
Unique key	Integer session ID; no two stored sessions share an ID.
Objectives	Add valid attempts, view every stored attempt, search by ID, and explain the CA1 OOP concepts.

Data dictionary

Member	Type	Meaning and validation
id	int	Session key: 1-9999, unique. Example: 101.
speaker	std::string	Speaker name. Must contain at least one non-whitespace character.
topic	std::string	Speech topic. Must contain at least one non-whitespace character.
durationSec	int	Practice duration: 30-1800 seconds inclusive.
words	int	Total spoken words, including fillers: 1-5000 inclusive.
fillers	int	Number of filler words: 0 through the total word count.
count	static int	Shared populated-record count; starts at 0 and is limited to 50.

Inputs, output and storage

The menu accepts 1 (Add Session), 2 (View All), 3 (Search by ID) and 0 (Exit). Output shows the original details, words per minute, filler percentage, and simple feedback labels. A local array holds up to 50 records; only its populated slots are read.

This CA1 evaluator uses manual counts. It does not capture audio or judge pronunciation, confidence, content quality, or delivery technique.

CA1 / PROJECT REPORT

2. UML and class features

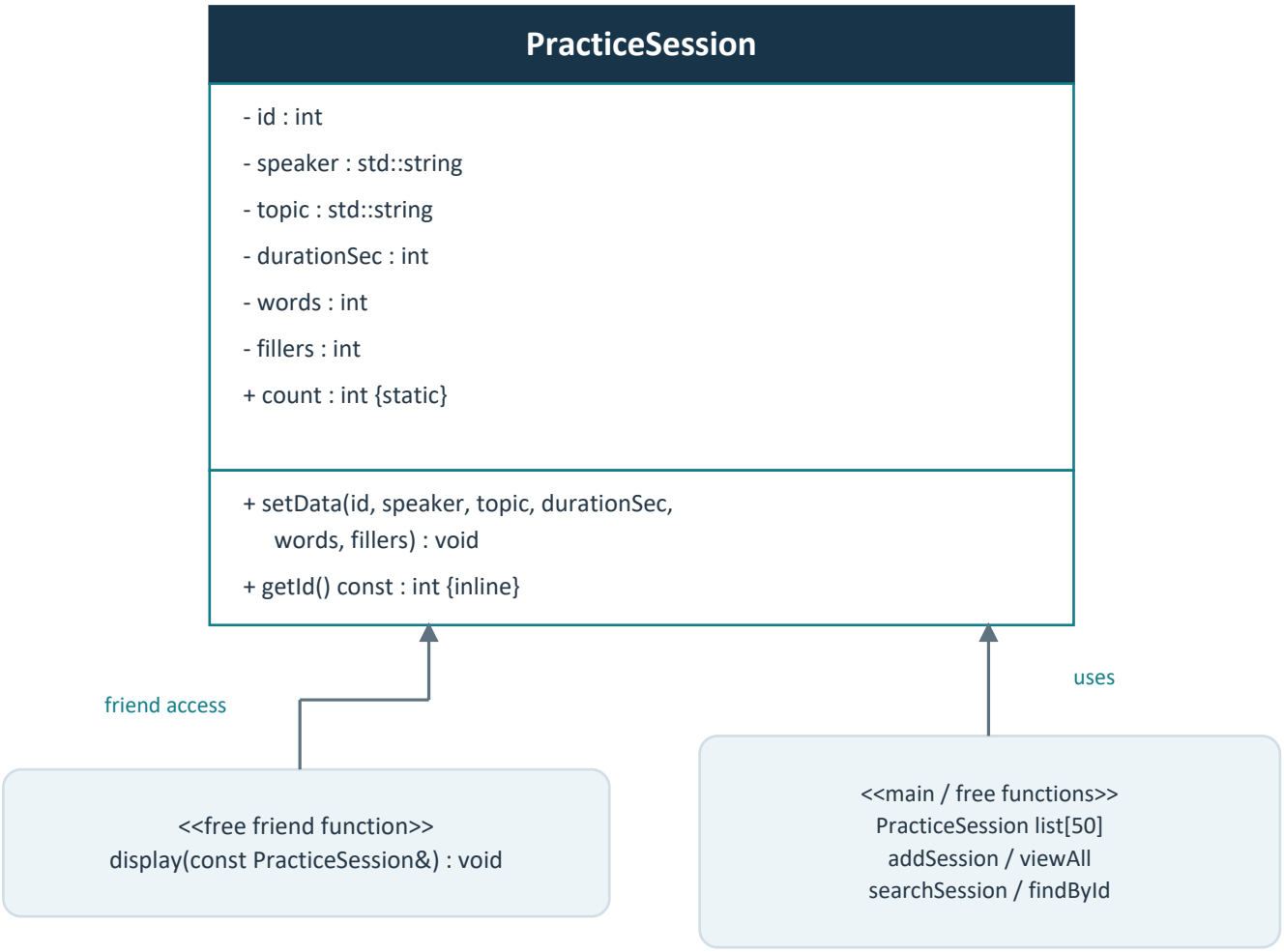


Figure 1. Private members use -, public members use +. display is a non-member friend. main owns the array; count tracks 0-50 populated slots.

CA1 feature	Evidence in src/main.cpp
Entity and encapsulation	PracticeSession, lines 11-32; six private record fields.
static	count declaration at line 20; shared definition at line 34.
inline	getId() const at line 30. Actual machine-code inlining is a compiler decision.
friend	Declaration at line 31; display at lines 36-47 reads private data.
this pointer	setData, lines 21-29, distinguishes members from matching parameter names.

The complete, line-numbered source is on pages 7-9. setData assigns every field before the shared populated-record count increases.

CA1 / PROJECT REPORT

3. Flowchart and Create algorithm

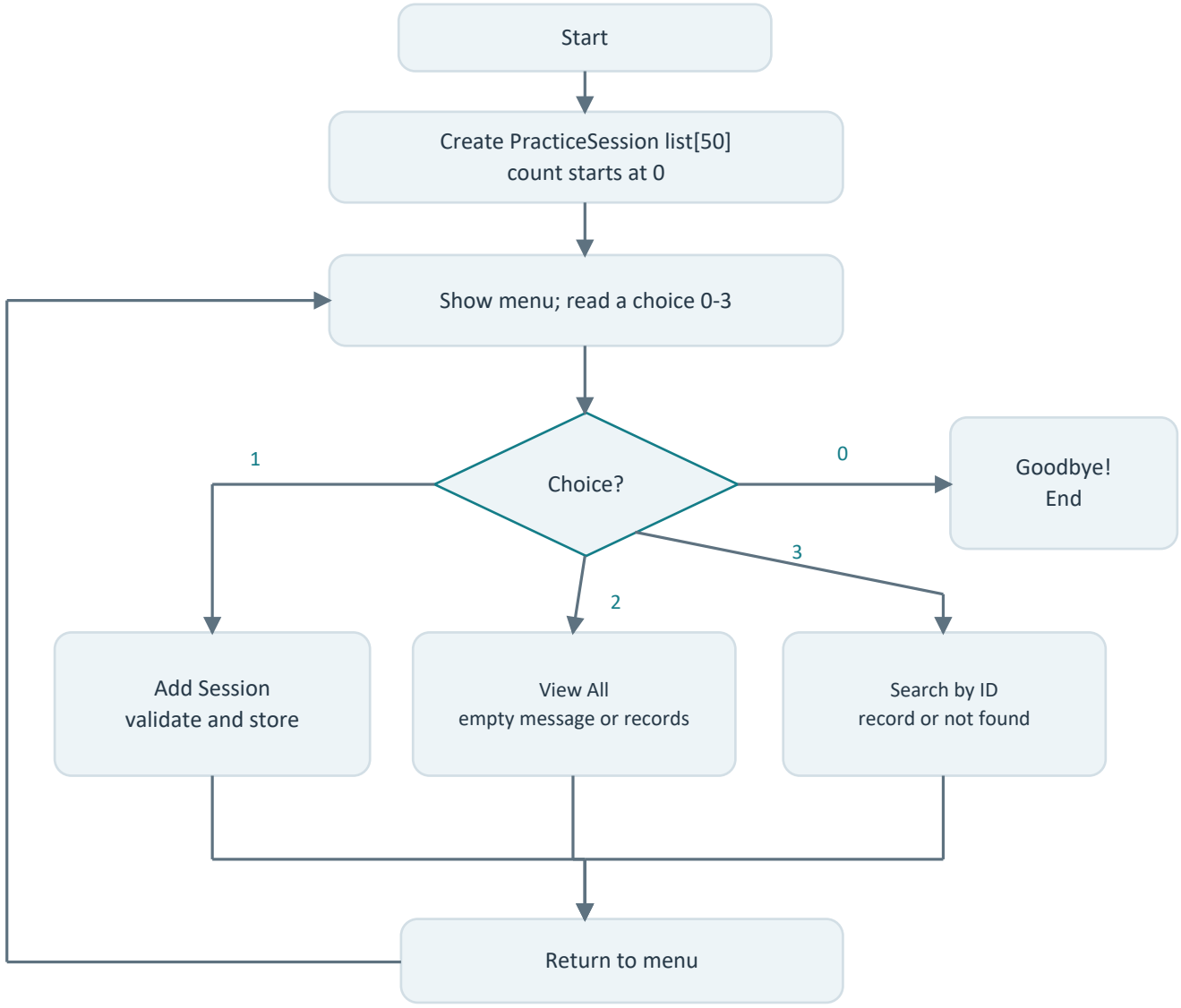


Figure 2. Invalid menu input is requested again. Choice 0 or end-of-input ends the main loop. Each completed menu action returns to the menu.

Add Session: step-by-step algorithm

1. If count is 50, report that the log is full and return. **2.** Read a valid session ID; use findById to reject a duplicate. **3.** Read nonblank speaker/topic text and valid duration, word and filler counts. **4.** Store all fields in list[count] using setData. **5.** Increase count and print Session added.

A field error re-prompts the same field. A full log or duplicate ID returns to the menu. If input ends during entry, the incomplete record is not stored.

CA1 / PROJECT REPORT

4. Create, Read and evaluation logic

Arrays, pointers and strings

Concept	Implementation and purpose
Array of objects	PracticeSession list[MAX_SESSIONS] at line 123 reserves 50 slots. Valid records occupy indices 0 through count - 1.
Pointer lookup	findById, lines 74-79, scans IDs and returns &list[i], or nullptr if no record matches.
Pointer traversal	viewAll, lines 103-112, starts p at the array, prints *p and advances p for each populated record.
Strings and input	speaker/topic use std::string and getline. readText rejects blank or spaces-only input; readInt parses a complete line and checks the range.

Create and Read operations

Create: addSession validates capacity, key uniqueness and every field before committing the record. **Read all:** viewAll displays records with feedback or prints No practice sessions yet. **Read one:** searchSession validates an ID and displays the pointed-to record or Session not found.

Evaluation formulas and project rules

```
wordsPerMinute = words * 60.0 / durationSec
fillerPercent  = fillers * 100.0 / words
```

Measure	Feedback rule
Speaking rate	Below 110 wpm: Slow; 110-160 inclusive: Balanced; above 160: Fast.
Filler-word rate	At most 5%: Controlled; above 5%: Reduce fillers.

The feedback labels use project-defined practice thresholds. Classification uses the full calculated values; displayed rates are rounded to one decimal place. Positive duration and word-count ranges prevent division by zero.

Worked example and complexity

For 260 words in 120 seconds with 8 fillers: rate = 130.0 wpm; filler rate = 3.1%. The output is Balanced / Controlled. An ID lookup or duplicate check takes O(n) time; listing takes O(n) record visits plus text output. Array traversal uses O(1) extra space.

Current limits: 50 sessions, manual data entry, and records lost on exit. count is designed for the single array used by main.

CA1 / PROJECT REPORT

5. CA1 testing and results

Compiled and executed on 06 November 2026 using MinGW g++ 6.3.0 with -std=c++11 -Wall -Wextra -pedantic. The compiler produced no diagnostics. Each case below passed against the speaking-practice implementation.

Test	Type	Scenario	Observed result: PASS
T01	Boundary	View empty log	Empty-log message.
T02	Invalid	Search empty log	Session not found.
T03	Normal	Add a valid session	Stored; 130.0 wpm, 3.1%, Balanced.
T04	Invalid	Duplicate session ID	Duplicate rejected; count unchanged.
T05	Boundary	ID 0 and 10000	Both IDs rejected; valid ID accepted.
T06	Invalid	ID abc, 12x and 1.5	All three invalid values rejected.
T07	Invalid	Empty / spaces-only text	Blank and spaces-only text rejected.
T08	Boundary	Duration 29 and 1801	Both durations rejected.
T09	Boundary	Words 0 and 5001	Both word counts rejected.
T10	Boundary	Fillers -1 and words + 1	Both filler counts rejected.
T11	Normal	View two populated records	Both records displayed.
T12	Normal	Search existing ID 102	ID 102 displayed with Reduce fillers.
T13	Invalid	Search missing ID 999	Session not found.
T14	Invalid	Menu 9, then Exit	Invalid choice rejected; normal exit.
T15	Boundary	50 records and attempted 51st	50 adds; 51st rejected; ID 50 retained.
T16	Boundary	Inclusive min/max values	IDs 1/9999 and all numeric endpoints accepted.
T17	Boundary	Pace and filler thresholds	110/160 wpm and 5% handled inclusively.

Captured output: search session 102

```
Session 102 | Saurabh kumar | Importance of reading
Duration: 90s | Words: 180 | Fillers: 12
Rate: 120.0 wpm | Filler rate: 6.7%
Pace: Balanced | Fillers: Reduce fillers
```

All 17 documented cases passed. Empty and interrupted input also exited cleanly without storing a partial record. Sample data and per-case input/output logs accompany the source and are embedded in this PDF.

CA1 IMPLEMENTATION / src/main.cpp

6. Complete source code (1/3)

Original source lines 1-47. The left-hand numbers are report annotations; all program lines are reproduced.

```
001  // Public Speaking Practice Evaluator - CAP455 CA1 (Units I-II)
002  // Manual practice records, simple feedback, in-memory Create and Read.
003  #include <iostream>
004  #include <iomanip>
005  #include <string>
006  #include <sstream>
007  using namespace std;
008
009  const int MAX_SESSIONS = 50;
010
011  class PracticeSession {
012  private:
013      int id;
014      string speaker;
015      string topic;
016      int durationSec;
017      int words;
018      int fillers;
019  public:
020      static int count;
021      void setData(int id, string speaker, string topic,
022                   int durationSec, int words, int fillers) {
023          this->id = id;
024          this->speaker = speaker;
025          this->topic = topic;
026          this->durationSec = durationSec;
027          this->words = words;
028          this->fillers = fillers;
029      }
030      inline int getId() const { return id; }
031      friend void display(const PracticeSession &s);
032  };
033
034  int PracticeSession::count = 0;
035
036  void display(const PracticeSession &s) {
037      double wpm = s.words * 60.0 / s.durationSec;
038      double fillerPercent = s.fillers * 100.0 / s.words;
039      string pace = wpm < 110 ? "Slow" : (wpm > 160 ? "Fast" : "Balanced");
040      string feedback = fillerPercent <= 5 ? "Controlled" : "Reduce fillers";
041      cout << "Session " << s.id << " | " << s.speaker << " | " << s.topic << '\n';
042      cout << "Duration: " << s.durationSec << "s | Words: " << s.words
043           << " | Fillers: " << s.fillers << '\n';
044      cout << fixed << setprecision(1) << "Rate: " << wpm
045           << " wpm | Filler rate: " << fillerPercent << "%\n";
046      cout << "Pace: " << pace << " | Fillers: " << feedback << '\n';
047  }
```

This part defines the entity, static count, this-based setter, inline getter and friend display function. The feedback values are calculated when an existing record is read.

CA1 IMPLEMENTATION / src/main.cpp

7. Complete source code (2/3)

Original source lines 48-101. The left-hand numbers are report annotations; all program lines are reproduced.

```
048
049 // Whole-line integer parsing rejects non-numeric suffixes and decimals.
050 bool readInt(const string &prompt, int low, int high, int &value) {
051     string line;
052     while (cout << prompt, getline(cin, line)) {
053         istringstream input(line);
054         int candidate;
055         char extra;
056         if (input >> candidate && !(input >> extra)
057             && candidate >= low && candidate <= high) {
058             value = candidate;
059             return true;
060         }
061         cout << "Invalid: enter an integer from " << low << " to " << high << ".\n";
062     }
063     return false;
064 }
065
066 bool readText(const string &prompt, string &value) {
067     while (cout << prompt, getline(cin, value)) {
068         if (value.find_first_not_of(" \t\r\n") != string::npos) return true;
069         cout << "Text cannot be blank.\n";
070     }
071     return false;
072 }
073
074 PracticeSession* findById(PracticeSession list[], int id) {
075     for (int i = 0; i < PracticeSession::count; ++i) {
076         if (list[i].getId() == id) return &list[i];
077     }
078     return nullptr;
079 }
080
081 void addSession(PracticeSession list[]) {
082     if (PracticeSession::count >= MAX_SESSIONS) {
083         cout << "Practice log is full.\n";
084         return;
085     }
086     int id, seconds, words, fillers;
087     string speaker, topic;
088     if (!readInt("Session ID (1-9999): ", 1, 9999, id)) return;
089     if (findById(list, id)) {
090         cout << "Error: session ID " << id << " already exists.\n";
091         return;
092     }
093     if (!readText("Speaker name: ", speaker)) return;
094     if (!readText("Speech topic: ", topic)) return;
095     if (!readInt("Duration in seconds (30-1800): ", 30, 1800, seconds)) return;
096     if (!readInt("Word count (1-5000): ", 1, 5000, words)) return;
097     if (!readInt("Filler count (0-word count): ", 0, words, fillers)) return;
098     list[PracticeSession::count].setData(id, speaker, topic, seconds, words, fillers);
099     ++PracticeSession::count;
100     cout << "Session added.\n";
101 }
```

CA1 IMPLEMENTATION / src/main.cpp

8. Complete source code (3/3)

Original source lines 102-137. The left-hand numbers are report annotations; all program lines are reproduced.

```
102
103  void viewAll(PracticeSession list[]) {
104      if (PracticeSession::count == 0) {
105          cout << "No practice sessions yet.\n";
106          return;
107      }
108      PracticeSession *p = list;
109      for (int i = 0; i < PracticeSession::count; ++i, ++p) {
110          display(*p);
111      }
112  }
113
114  void searchSession(PracticeSession list[]) {
115      int id;
116      if (!readInt("Session ID to search: ", 1, 9999, id)) return;
117      PracticeSession *p = findById(list, id);
118      if (p) display(*p);
119      else cout << "Session not found.\n";
120  }
121
122  int main() {
123      PracticeSession list[MAX_SESSIONS];
124      int choice;
125      while (true) {
126          cout << "\n=== Public Speaking Practice Evaluator ===\n"
127               << "1. Add Session\n2. View All\n3. Search by ID\n0. Exit\n";
128          if (!readInt("Choice: ", 0, 3, choice) || choice == 0) break;
129          switch (choice) {
130              case 1: addSession(list); break;
131              case 2: viewAll(list); break;
132              case 3: searchSession(list); break;
133          }
134      }
135      cout << "Goodbye!\n";
136      return 0;
137  }
```

Implementation summary

The main function manages the menu and dispatches each operation. Record creation increases the shared count only after validation succeeds. View All traverses the populated array through an object pointer. Search returns the matching record address or nullptr. The friend display function calculates and presents the speaking-rate and filler-word feedback.

CA1 / PROJECT REPORT

9. Execution results and conclusion

Execution environment

The application was compiled as C++11 with MinGW g++ 6.3.0 on Windows. Compilation completed without warnings. The sample execution used three practice sessions for the speaker Saurabh kumar.

```
g++ -std=c++11 -Wall -Wextra -pedantic src/main.cpp -o speaking.exe
Get-Content data\sample_input.txt | .\speaking.exe
```

Sample input records

ID	Speech topic	Seconds	Words	Fillers
101	Self introduction	120	260	8
102	Importance of reading	90	180	12
103	Interview introduction	60	180	4

Calculated results

ID	Rate (wpm)	Fillers (%)	Pace	Filler feedback
101	130.0	3.1	Balanced	Controlled
102	120.0	6.7	Balanced	Reduce fillers
103	180.0	2.2	Fast	Controlled

All three records were created and displayed successfully. Searching for ID 102 returned its stored details and calculated feedback. The program exited normally after the sample run.

Console output: session 103

```
Session 103 | Saurabh kumar | Interview introduction
Duration: 60s | Words: 180 | Fillers: 4
Rate: 180.0 wpm | Filler rate: 2.2%
Pace: Fast | Fillers: Controlled
```

Conclusion

The Public Speaking Practice Evaluator provides an organised in-memory record of speaking attempts and consistent feedback based on duration, word count and filler usage. The implementation demonstrates encapsulation, static data, an inline function, a friend function, the this pointer, strings, an object array and pointer-based record access.

Validation prevents duplicate IDs and invalid record values. All 17 documented normal, invalid and boundary cases passed, including the 50-record capacity check. The implemented functionality meets the CA1 Design & Create scope.

References

1. CAP455 Minor Project Quick Guide, Session 2026-27: CA1 requirements.
2. Public Speaking Practice Evaluator: source code, sample data and test execution records.

Acknowledgement

AI assistance (OpenAI Codex) was used for implementation, testing and report preparation.